Revision Notes 03–04: What a Matrix Can Reach, What It Forgets, and How a Machine Learns from Its Neighbours

AI 501: Mathematics for Artificial Intelligence

Fall 2026

Abstract

These notes revise two lectures that look unrelated but are really one story. In **Part I** we treat a matrix as a *machine*. The machine turns input recipes into outputs, and we ask three questions: *what can it produce* (column space), *how many genuinely different things can it produce* (rank), and *which inputs does it make invisible* (null space). In **Part II** the matrix becomes a table of patients, and the machine becomes a learned model. We ask how to measure its mistakes (loss functions) and how to be sure it has learned rather than memorised (train/validation/test). We then build the simplest learner there is, k-nearest neighbours, and finish with an honest way to grade a classifier (confusion matrix, ROC/AUC, F1, MCC). Along the way there are *brainteasers* for you to try. Their solutions are collected at the end, so try each one before you peek.

How to read these notes. **Orange** boxes tell a story or analogy, **blue** boxes hold the one sentence you should remember, **red** boxes warn about traps, **green** boxes connect the maths to AI, **purple** boxes are brainteasers, and **grey** boxes point out places where the slides are shorthand for something more precise. Material marked *Beyond the slides* fills small gaps and will help you, but it is extra.

Contents

Part I Matrices: What They Reach and What They Forget **2**

1 The paint-mixing machine **2**

2 Rank: counting the genuinely new directions **3**

2.1 Redundant ingredients . . . 3

2.2 The geometric picture . . . 3

2.3 Computing rank by row reduction . . . 3

2.4 Three key properties, and the intuition behind each . . . 5

3 Column space: the world a matrix can reach **5**

4 Null space: the directions a matrix forgets **6**

4.1 The rank–nullity theorem: conservation of dimensions . . . 8

5 Putting it together: when does $A\mathbf{x} = \mathbf{b}$ have solutions? **9**

Part II Learning from Examples **10**

6 The supervised learning setup **10**
6.1 Data, features and labels . . . 10
6.2 The unknown distribution and the hypothesis . . . 11

7 Loss functions: putting a number on "wrong" **11**

8 Learning, overfitting and the train/test split **13**
8.1 Learning is optimisation . . . 13
8.2 The memoriser: the first glimpse of overfitting . . . 13
8.3 The fix: hold data back . . . 13
8.4 Validation: the mock exam . . . 14

9 A map of classifiers **14**

10 k-Nearest Neighbours: "tell me who your friends are" **15**
10.1 The idea . . . 15
10.2 Formal definition . . . 15
10.3 What kind of learner is k-NN? . . . 16
10.4 Measuring similarity: distance metrics . . . 16
10.5 Choosing k: between paranoia and blindness . . . 18
10.6 Practical issues . . . 18
10.7 Storage and time complexity . . . 19

11 Grading a classifier honestly **20**
11.1 Accuracy and its trap . . . 20
11.2 TP, TN, FP, FN . . . 20
11.3 The confusion matrix . . . 20
11.4 The metrics: rows versus columns . . . 21
11.5 When to use which? . . . 23
11.6 Trade-offs: the threshold slider . . . 23
11.7 The ROC curve and AUC . . . 23
11.8 F1-score: one number for precision and recall . . . 24
11.9 Matthews correlation coefficient (MCC) . . . 25

12 Summary: the big picture **26**

Solutions to the brainteasers **28**

Part I Matrices: What They Reach and What They Forget

1 The paint-mixing machine

Story / analogy: the paint shop

Imagine a paint shop with a mixing machine. The machine has three tubes of base paint loaded into it, and each tube is described by how much *Red, Green, Blue* light its colour reflects:

$$\underbrace{\mathbf{a}_1 = \begin{pmatrix}1\\0\\0\end{pmatrix}}_{\text{red}}, \qquad \underbrace{\mathbf{a}_2 = \begin{pmatrix}0\\0\\1\end{pmatrix}}_{\text{blue}}, \qquad \underbrace{\mathbf{a}_3 = \begin{pmatrix}1\\0\\1\end{pmatrix}}_{\text{purple}}.$$

A customer hands over a *recipe* $\mathbf{x} = (x_1, x_2, x_3)^\top$: "$x_1$ units of red, x_2 of blue, x_3 of purple." The machine outputs the colour

$$A\mathbf{x} \;=\; x_1\mathbf{a}_1 + x_2\mathbf{a}_2 + x_3\mathbf{a}_3, \qquad A = \begin{pmatrix}\mathbf{a}_1 & \mathbf{a}_2 & \mathbf{a}_3\end{pmatrix} = \begin{pmatrix}1 & 0 & 1\\ 0 & 0 & 0\\ 0 & 1 & 1\end{pmatrix}.$$

(Mathematically we also allow negative amounts. Think of them as "remove that much of this paint".)

This is the *column picture* of matrix–vector multiplication from Notes 02, and it is the lens for all of Part I:

Key idea

A matrix $A \in \mathbb{R}^{m\times n}$ is a machine with n input knobs. The columns $\mathbf{a}_1, \dots, \mathbf{a}_n \in \mathbb{R}^m$ are its *ingredients*, the vector $\mathbf{x} \in \mathbb{R}^n$ is the *recipe*, and $A\mathbf{x} \in \mathbb{R}^m$ is the *product*.

$$\mathbf{x} \in \mathbb{R}^n \;\xrightarrow{\;A\;}\; A\mathbf{x} \in \mathbb{R}^m.$$

Three natural questions about the paint shop will turn into the three big ideas of this part:

Q1. What colours can the shop make at all? (Can it make yellow?) $\Rightarrow$ *column space.*

Q2. How many genuinely different base paints does it have? (Purple is just red + blue.) $\Rightarrow$ *rank.*

Q3. Which recipes produce nothing at all? $\Rightarrow$ *null space.*

Brainteaser 1 solution →

Before reading on, use only your intuition. (a) Can this machine ever produce yellow, $(1, 1, 0)^\top$? (b) Find a non-zero recipe $\mathbf{x}$ with $A\mathbf{x} = \mathbf{0}$, i.e. a recipe that uses paint but produces "no colour change at all".

2 Rank: counting the genuinely new directions

2.1 Redundant ingredients

The purple tube is a fake. Anything you can mix with purple you could already mix with red and blue, since $\mathbf{a}_3 = \mathbf{a}_1 + \mathbf{a}_2$. The shop *looks* like it has three ingredients, but it really has only **two independent directions**. That number is the rank.

Definition: rank

- **Column rank:** the maximum number of linearly independent columns of A.
- **Row rank:** the maximum number of linearly independent rows of A.
- **Rank:** these two numbers are *always equal* (a surprising fact, explained in Section 2.3), and their common value is $\text{rank}(A)$.

For every $A \in \mathbb{R}^{m \times n}$,

$$\boxed{\text{rank}(A) \leq \min(m, n)}$$

Reading the slides carefully

The slide defines rank as "the minimum of the number of linearly independent rows and the number of linearly independent columns". That is correct, but there is no real contest to take the minimum of, because the two numbers are always the same. The inequality $\text{rank}(A) \leq \min(m, n)$ comes from something else. You cannot have more independent columns than there are columns (n), and you cannot have more independent vectors in $\mathbb{R}^m$ than the dimension m.

Story / analogy: why $\min(m, n)$?

Few ingredients: three tubes can never give you four independent colour directions ($\text{rank} \leq n$). *Small world:* colours live in 3-D RGB space, so even with 100 tubes of paint you can never find more than 3 independent directions ($\text{rank} \leq m$). Rank is limited by whichever runs out first: ingredients or room.

2.2 The geometric picture

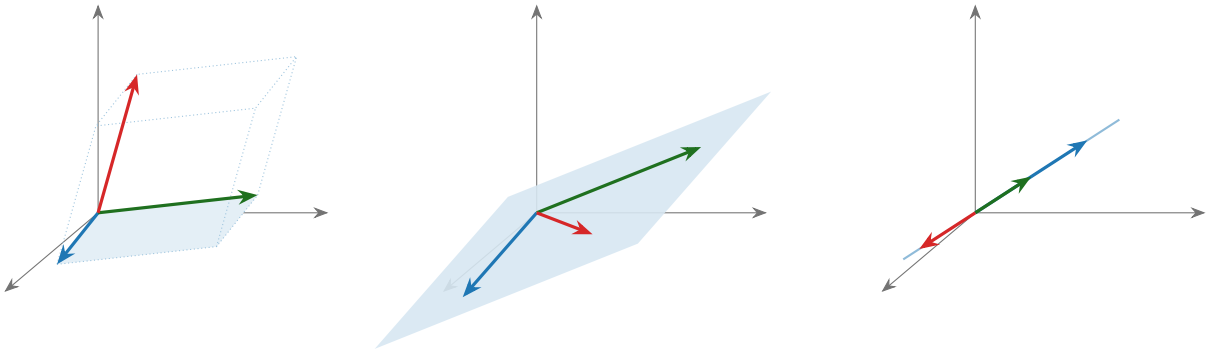
The columns of A are arrows in $\mathbb{R}^m$. The rank is the *dimension of the region those arrows can fill up* when you combine them. For a 3×3 matrix:

Key idea

Rank = the number of *genuinely different directions* a matrix can push you in. The number of columns tells you how many knobs you have. The rank tells you how many of those knobs actually do something new.

2.3 Computing rank by row reduction

Counting independent columns by eye stops working beyond small examples. The reliable method is **Gaussian elimination**: reduce A to row echelon form (REF) and count the non-zero rows, which is the same as counting the pivots.



rank 3: columns fill $\mathbb{R}^3$ (a solid volume)

rank 2: all columns lie on one plane

rank 1: all columns lie on one line

Figure 1: Rank is the dimension of what the columns can fill. Three arrows can span a volume, a plane, or only a line. (In the *Essence of Linear Algebra* language of 3Blue1Brown, rank is the number of dimensions left *after* the matrix squishes space.)

Worked example (from the slides).

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 1 & 1 \end{pmatrix} \xrightarrow[R_3 \leftarrow R_3 - R_1]{R_2 \leftarrow R_2 - 2R_1} \begin{pmatrix} 1 & 2 & 3 \\ 0 & 0 & 0 \\ 0 & -1 & -2 \end{pmatrix} \xrightarrow[R_2 \leftarrow -R_2]{\text{swap } R_2, R_3} \begin{pmatrix} \boxed{1} & 2 & 3 \\ 0 & \boxed{1} & 2 \\ 0 & 0 & 0 \end{pmatrix}.$$

There are two non-zero rows (two pivots), so $\operatorname{rank}(A) = 2$. Row 2 of A was just twice row 1, so it carried no new information, and elimination wiped it out.

Story / analogy: why row operations don't change the rank

Think of each row as one *equation* or one *piece of evidence*. Scaling an equation, swapping two equations, or adding a multiple of one to another reorganises the evidence, but it never creates new evidence and never destroys any. Elimination simply exposes redundancy: a row that was secretly a combination of the others turns into a row of zeros.

Why row rank = column rank (the pivot argument). Look at the final echelon form. Each pivot sits in its own row *and* its own column.

- The non-zero rows (one per pivot) are independent, so *row rank* = number of pivots.
- The pivot columns are independent, and every non-pivot column is a combination of them, so *column rank* = number of pivots. (Row operations don't change which column combinations equal zero, so this carries back to the original A.)

Both ranks are the same count of pivots. This also explains $\operatorname{rank}(A) = \operatorname{rank}(A^\top)$: transposing swaps rows and columns, and we just showed that swapping them does not change the count.

Going one step further to the *reduced* form (RREF) makes the redundancy explicit:

$$\operatorname{rref}(A) = \begin{pmatrix} 1 & 0 & -1 \\ 0 & 1 & 2 \\ 0 & 0 & 0 \end{pmatrix} \quad \Longrightarrow \quad \text{column } 3 = -1 \cdot (\text{column } 1) + 2 \cdot (\text{column } 2).$$

Check in the original: $-\begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix} + 2\begin{pmatrix} 2 \\ 4 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 6 \\ 1 \end{pmatrix}$. ✓ The RREF reads out the recipe for the redundant column.

Brainteaser 2 solution →

Without doing any elimination, give the rank of each matrix, and say which of the pictures in Figure 1 each one corresponds to.

$$B_1 = \begin{pmatrix} 2 & 4 & -6 \\ 1 & 2 & -3 \\ -3 & -6 & 9 \end{pmatrix}, \qquad B_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \qquad B_3 = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{pmatrix}, \qquad B_4 = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}.$$

2.4 Three key properties, and the intuition behind each

(P1) $\operatorname{rank}(A) = \operatorname{rank}(A^\top)$. This follows from the pivot argument above: the rows and the columns of a matrix hold "the same amount of independent information".

(P2) $\operatorname{rank}(AB) \le \min\big(\operatorname{rank}(A), \operatorname{rank}(B)\big)$.

Story / analogy: the pipeline

$AB\mathbf{x}$ means "first run $\mathbf{x}$ through machine B, then feed the result into machine A".

- Everything that comes out is an output of A, so it lies in the space A can reach, and $\operatorname{rank}(AB) \le \operatorname{rank}(A)$.
- B squeezes its inputs down to at most $\operatorname{rank}(B)$ dimensions, and a linear machine A can bend or shrink what it receives but can never *create* new dimensions, so $\operatorname{rank}(AB) \le \operatorname{rank}(B)$.

A pipeline is only as wide as its narrowest pipe.

(P3) If $A \in \mathbb{R}^{n\times n}$ is invertible, then $\operatorname{rank}(A) = n$. An invertible machine must be undoable. If it flattened even one direction, two different inputs would give the same output, and no one could tell which input to go back to. So nothing is flattened, and all n directions survive. (This matches Notes 02: $\det A \neq 0 \iff$ no squishing $\iff$ full rank.)

Brainteaser 3 solution →

Let $\mathbf{u} \in \mathbb{R}^5$ and $\mathbf{v} \in \mathbb{R}^4$ be non-zero. What is the size and the rank of the outer product $\mathbf{u}\mathbf{v}^\top$? Use property (P2) to justify your answer in one line.

3 Column space: the world a matrix can reach

Back to question Q1: *which colours can the shop produce?* The answer is the set of all mixtures of the columns.

Definition: column space

For $A \in \mathbb{R}^{m\times n}$ with columns $\mathbf{a}_1, \dots, \mathbf{a}_n \in \mathbb{R}^m$,

$$\mathcal{C}(A) = \operatorname{span}(\mathbf{a}_1, \dots, \mathbf{a}_n) = \{\, A\mathbf{x} \;:\; \mathbf{x} \in \mathbb{R}^n \,\}.$$

It lives in the *output* space $\mathbb{R}^m$, it is a subspace of $\mathbb{R}^m$, and

$$\dim \mathcal{C}(A) = \text{number of independent columns} = \operatorname{rank}(A).$$

The two descriptions say the same thing from two angles. "Span of the columns" says *everything you can mix*; "$\{A\mathbf{x}\}$" says *everything the machine can output.* The column space is the **menu** of the shop.

Why is it a subspace? Mixing two mixtures gives another mixture: $A\mathbf{x} + A\mathbf{y} = A(\mathbf{x}+\mathbf{y})$ and $c\,A\mathbf{x} = A(c\mathbf{x})$. The recipe $\mathbf{x} = \mathbf{0}$ gives $\mathbf{0}$, so the origin is always on the menu. Geometrically, a column space is always a flat object (a point, line, plane, ...) *passing through the origin.*

For our paint shop, $\mathcal{C}(A) = \text{span}\{(1,0,0)^\top, (0,0,1)^\top\} = \{(r,0,b)^\top\}$, the "red–blue plane". Every colour on the menu has zero green, so yellow $(1,1,0)^\top$ is off the menu.

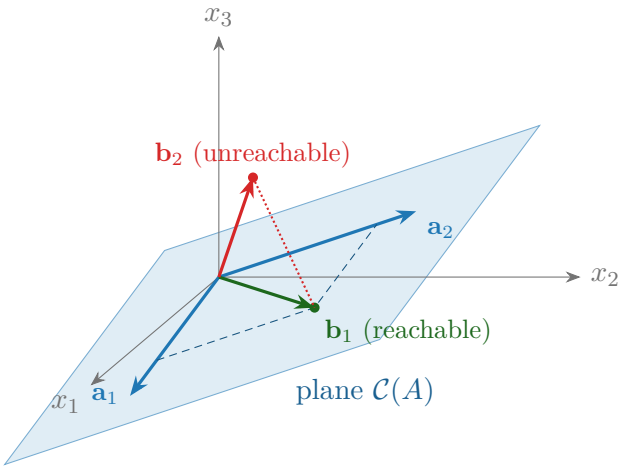


Figure 2: A 3×2 matrix has two columns in $\mathbb{R}^3$, and their column space is a plane through the origin. $A\mathbf{x} = \mathbf{b}_1$ has a solution because $\mathbf{b}_1$ is on the plane (dashed lines show the recipe $0.7\mathbf{a}_1 + 0.8\mathbf{a}_2$). $A\mathbf{x} = \mathbf{b}_2$ has *no* solution, because no recipe leaves the plane. The dotted segment is the gap between $\mathbf{b}_2$ and the plane. Closing that gap as well as possible is exactly what least-squares regression will do.

Key idea

$A\mathbf{x} = \mathbf{b}$ has a solution *if and only if* $\mathbf{b} \in \mathcal{C}(A)$. Solving a linear system means asking whether the target is on the menu.

The slide example.

$$A = \begin{pmatrix} 1 & 1 & 2 \\ 2 & 1 & 3 \\ 3 & 1 & 4 \\ 4 & 1 & 5 \end{pmatrix}, \qquad m = 4,\ n = 3.$$

The columns live in $\mathbb{R}^4$, so $\mathcal{C}(A)$ is a subspace of $\mathbb{R}^4$. The third column is the sum of the first two, so only two columns are independent, and $\mathcal{C}(A)$ is a **2-dimensional plane sitting inside 4-dimensional space**. We can't draw $\mathbb{R}^4$, but the picture in Figure 2 is still the right mental image: a flat sheet through the origin in a bigger room.

Brainteaser 4 solution →

For the 4×3 matrix above, decide whether each vector is on the menu: (a) $\mathbf{b} = (1,1,1,1)^\top$; (b) $\mathbf{b} = (3,5,7,9)^\top$; (c) $\mathbf{b} = (0,0,0,1)^\top$. *Hint:* two of these need almost no computation.

4 Null space: the directions a matrix forgets

Now question Q3: which recipes produce nothing?

In the paint shop, take one unit of red, one unit of blue, and *remove* one unit of purple:

$$A\begin{pmatrix} 1 \\ 1 \\ -1 \end{pmatrix} = \mathbf{a}_1 + \mathbf{a}_2 - \mathbf{a}_3 = \mathbf{0}.$$

You did something to the recipe, and the output did not change at all. The recipe $(1, 1, -1)^\top$ is a *ghost*: the machine cannot see it.

Definition: null space

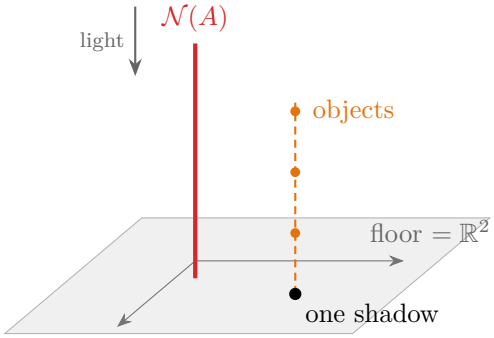
For $A \in \mathbb{R}^{m \times n}$,

$$\mathcal{N}(A) = \{\, \mathbf{x} \in \mathbb{R}^n \;:\; A\mathbf{x} = \mathbf{0} \,\}.$$

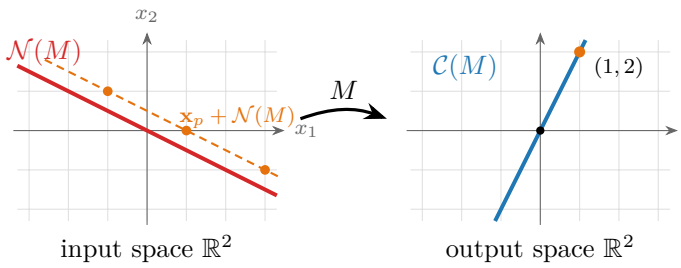
It lives in the *input* space $\mathbb{R}^n$ and is a subspace of $\mathbb{R}^n$. Its dimension is the **nullity** of A. In words, it is everything the machine sends to zero, or equivalently the *inverse image* of $\mathbf{0} \in \mathbb{R}^m$.

Story / analogy: shadows

Hold an object under a lamp that shines straight down. The lamp is a machine $\mathbb{R}^3 \to \mathbb{R}^2$: it maps each point to its shadow on the floor. Now slide the object *straight up or down* along the light beam. The shadow does not move at all. The vertical direction is the null space of the shadow machine: movements in that direction are *invisible* in the output. (Figure 3a.)



(a) $A = \left(\begin{smallmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{smallmatrix}\right)$ drops the height. Every point on the orange line has the same shadow.



(b) $M = \left(\begin{smallmatrix} 1 & 2 \\ 2 & 4 \end{smallmatrix}\right)$ squishes the whole plane onto the blue line. The red line $\mathcal{N}(M) = \text{span}\{(2, -1)^\top\}$ collapses to the origin. The three orange inputs, which sit on a line *parallel* to $\mathcal{N}(M)$, all land on the same output $(1, 2)^\top$.

Figure 3: Two pictures of a null space. The matrix forgets movement along the red direction.

Computing a null space. Solve $A\mathbf{x} = \mathbf{0}$ using the RREF. For the rank example from Section 2.3:

$$\text{rref}(A) = \begin{pmatrix} 1 & 0 & -1 \\ 0 & 1 & 2 \\ 0 & 0 & 0 \end{pmatrix} \implies \begin{cases} x_1 - x_3 = 0 \\ x_2 + 2x_3 = 0 \end{cases} \implies \mathbf{x} = x_3 \begin{pmatrix} 1 \\ -2 \\ 1 \end{pmatrix}, \qquad \mathcal{N}(A) = \text{span}\left\{ \begin{pmatrix} 1 \\ -2 \\ 1 \end{pmatrix} \right\}.$$

The variable x_3 (the column without a pivot) is *free*. Every free variable adds one dimension to the null space. Notice that $(1, -2, 1)$ is exactly the column relation we found earlier, $\mathbf{a}_1 - 2\mathbf{a}_2 + \mathbf{a}_3 = \mathbf{0}$. Null space vectors and column dependencies are the same thing.

Key idea

Null-space vectors are the *recipes for redundancy*. Each one records a way the columns cancel each other out. If the columns are independent, the only such recipe is $\mathbf{x} = \mathbf{0}$, so $\mathcal{N}(A) = \{\mathbf{0}\}$.

For the 4×3 slide example, column 3 = column 1 + column 2, so $\mathcal{N}(A) = \text{span}\{(1, 1, -1)^\top\}$, a line in $\mathbb{R}^3$.

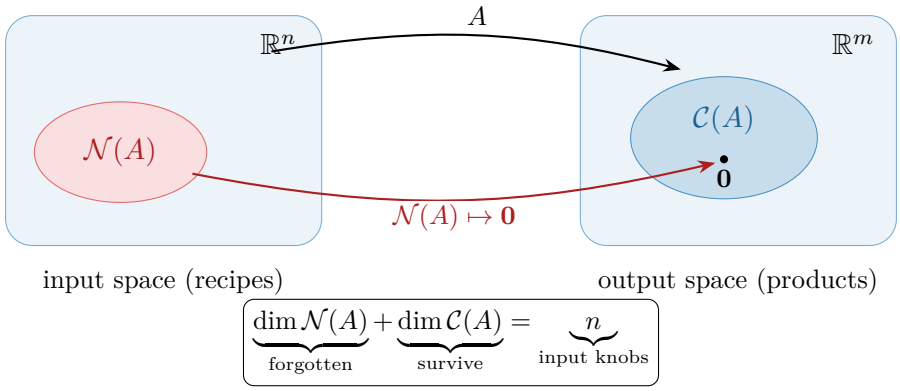


Figure 4: The null space lives in the input space, and the column space lives in the output space. Rank–nullity is a bookkeeping rule for the n input dimensions.

4.1 The rank–nullity theorem: conservation of dimensions

Key idea: rank–nullity theorem

$$\text{rank}(A) + \text{nullity}(A) = n \text{ (number of columns)}, \qquad \text{i.e.} \qquad \dim \mathcal{C}(A) + \dim \mathcal{N}(A) = n.$$

Story / analogy: a budget of dimensions

You start with n independent knobs, so the input space has n dimensions. The machine treats each input dimension in one of two ways:

- it **survives** and becomes a genuinely new direction in the output (these count toward the rank), or
- it is **crushed** to zero (these count toward the nullity).

Nothing disappears unaccounted for. Every lost output dimension shows up as a ghost dimension in the input.

Figure 4 puts the two spaces side by side. The null space $\mathcal{N}(A)$ is a region of the *input* space $\mathbb{R}^n$ whose every point the machine sends to $\mathbf{0}$, while the column space $\mathcal{C}(A)$ is the region of the *output* space $\mathbb{R}^m$ the machine can actually reach. Rank–nullity is the balance sheet between them: the n input dimensions split into the $\dim \mathcal{N}(A)$ that are forgotten and the $\dim \mathcal{C}(A) = \text{rank}(A)$ that survive.

Check it on every example so far.

matrix	n	rank	nullity	check
paint shop (3×3)	3	2	1	$2 + 1 = 3$ ✓
$\begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 1 & 1 \end{pmatrix}$	3	2	1	✓
slide example (4×3)	3	2	1	✓
$M = \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$	2	1	1	✓
shadow $\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$	3	2	1	✓

Common trap

The theorem counts the **columns** n, not the rows m. For the 4×3 slide example the ranks add up to 3, not 4. Also remember *where* each space lives: $\mathcal{N}(A) \subseteq \mathbb{R}^n$ (inputs) but $\mathcal{C}(A) \subseteq \mathbb{R}^m$ (outputs). You cannot compare a null-space vector with a column-space vector unless $m = n$.

Brainteaser 5 solution →

A matrix $W \in \mathbb{R}^{3\times 7}$ has rank 3. (a) What is $\dim \mathcal{N}(W)$? (b) Is $\mathcal{C}(W)$ all of $\mathbb{R}^3$? (c) Can a 7×3 matrix ever have a trivial null space? Can a 3×7 matrix?

5 Putting it together: when does $A\mathbf{x} = \mathbf{b}$ have solutions?

Column space and null space together answer everything about a linear system:

- **Existence** is a column-space question: is $\mathbf{b} \in \mathcal{C}(A)$?
- **Uniqueness** is a null-space question: if $\mathbf{x}_p$ is one solution, then so is $\mathbf{x}_p + \mathbf{z}$ for *every* $\mathbf{z} \in \mathcal{N}(A)$, because $A(\mathbf{x}_p + \mathbf{z}) = \mathbf{b} + \mathbf{0} = \mathbf{b}$.

$$\boxed{\text{all solutions of } A\mathbf{x} = \mathbf{b} \;=\; \mathbf{x}_p + \mathcal{N}(A)} \qquad \text{(one particular solution, plus any ghost recipe).}$$

This is exactly the dashed orange line in Figure 3b: a copy of $\mathcal{N}(M)$ shifted so that it passes through one solution. In the paint shop, the colour $(1,0,1)^\top$ can be mixed as $(1,1,0)^\top$, as $(0,0,1)^\top$, or as $(1,1,0)^\top + t(-1,-1,1)^\top$ for any t. There are infinitely many recipes because the shop has a ghost direction.

Reading the slides carefully: rank and linear systems

The slide lists three rules for $A \in \mathbb{R}^{m\times n}$. They are shorthand, and here is the precise version:

Slide says	Precisely
$\operatorname{rank}(A) = n \Rightarrow$ unique solution	$\mathcal{N}(A) = \{\mathbf{0}\}$, so there is *at most one* solution. It is unique *if* $\mathbf{b} \in \mathcal{C}(A)$.
$\operatorname{rank}(A) < n \Rightarrow$ infinitely many	$\mathcal{N}(A)$ is non-trivial, so *if* a solution exists there are infinitely many.
$\operatorname{rank}(A) < m \Rightarrow$ inconsistent	$\mathcal{C}(A) \neq \mathbb{R}^m$, so *some* targets $\mathbf{b}$ are unreachable. Others may still be reachable.

The exact test (beyond the slides): $A\mathbf{x} = \mathbf{b}$ is consistent $\iff \operatorname{rank}(A) = \operatorname{rank}([A \,|\, \mathbf{b}])$. If appending $\mathbf{b}$ as an extra column raises the rank, then $\mathbf{b}$ was a new direction, which means it was off the menu. This is why the slide's recipe for row reduction starts with "write the *augmented* matrix".

case	shape	$\mathcal{C}(A)$	solutions of $A\mathbf{x} = \mathbf{b}$
$r = m = n$	square, invertible	all of $\mathbb{R}^m$	exactly one, for every $\mathbf{b}$
$r = n < m$	tall, independent columns	a slice of $\mathbb{R}^m$	0 or 1
$r = m < n$	wide, independent rows	all of $\mathbb{R}^m$	infinitely many, for every $\mathbf{b}$
$r < m,\ r < n$	rank deficient	a slice	0 or infinitely many

Brainteaser 6 solution →

For $M = \left(\begin{smallmatrix} 1 & 2 \\ 2 & 4 \end{smallmatrix}\right)$: (a) Write down *all* solutions of $M\mathbf{x} = (3,6)^\top$. (b) How many solutions does $M\mathbf{x} = (3,5)^\top$ have? Answer using the picture in Figure 3b, not algebra.

Where this shows up in AI: why a machine-learning student should care

- **Regression is a column-space problem.** Stack n patients' features into a data matrix $X \in \mathbb{R}^{n\times d}$ and their outcomes into $\mathbf{y} \in \mathbb{R}^n$. A linear model predicts $X\boldsymbol{\theta}$, which is always a point in $\mathcal{C}(X)$. With many more patients than features ($n \gg d$), $\mathcal{C}(X)$ is a thin slice of $\mathbb{R}^n$, and $\mathbf{y}$ is almost never exactly on it. That is the unreachable $\mathbf{b}_2$ of Figure 2. The best we can do is the closest reachable point, and that is *least squares*, the picture on the slides' title page.
- **Redundant features create a null space.** Suppose one column is height in cm and another is height in inches. Then X has dependent columns and $\mathcal{N}(X) \neq \{\mathbf{0}\}$, so the weights $\boldsymbol{\theta}$ and $\boldsymbol{\theta} + t\mathbf{z}$ ($\mathbf{z} \in \mathcal{N}(X)$) give *identical predictions*. The model cannot decide how to split credit between the two columns. That is why we remove duplicated features or add regularisation.
- **Compression uses low rank.** A huge matrix of rank r can be stored with far fewer numbers (Brainteaser 3). This idea returns with the SVD, PCA and low-rank fine-tuning of large models.

Watch next (strongly recommended): 3Blue1Brown, *Essence of Linear Algebra*. Ch. 2: span & basis, Ch. 3: linear transformations, Ch. 7: inverse matrices, column space and null space.

Part II Learning from Examples

Story / analogy: the bridge between the two parts

In Part I a matrix was a machine we *knew*, and we asked what it could produce. In machine learning the situation is reversed. We see examples of inputs and outputs, and we have to *find* the machine. The data are still vectors and matrices: every patient is a point in $\mathbb{R}^d$, and a dataset is a matrix with one row per patient. Our first learning algorithm, k-NN, is built on nothing more than the *distance between vectors* from Notes 01.

6 The supervised learning setup

6.1 Data, features and labels

A clinic in Lahore has records for 200 patients. For each patient it has measured *features*: age, cholesterol, glucose, BMI, height, heart rate, calorie intake and daily steps. It wants to predict a *label*, for example the patient's blood-oxygen saturation.

Definition: supervised learning data

$$D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_n, y_n)\} \subseteq \mathcal{X}^d \times \mathcal{Y},$$

- $\mathbf{x}_i \in \mathcal{X}^d$ is the **feature vector** of example i, in a d-dimensional feature space (here $d = 8$ and $n = 200$);
- $y_i \in \mathcal{Y}$ is its **label**, the answer we would like to predict.

The type of label decides the type of problem:

- **Regression:** $\mathcal{Y} = \mathbb{R}$, a continuous prediction (oxygen saturation, house price).
- **Binary classification:** $\mathcal{Y} = \{0, 1\}$ or $\{-1, 1\}$ or $\{1, 2\}$ (disease or no disease).
- **M-class classification:** $\mathcal{Y} = \{1, 2, \ldots, M\}$ (which of M digits is in the image?).

Brainteaser 7 solution →

Regression or classification? (a) Predicting tomorrow's maximum temperature in Lahore. (b) Predicting whether a bowler's next delivery is a no-ball. (c) Predicting a patient's blood group. (d) Predicting the number of runs scored off the next over. (Part (d) is debatable. Say what you would choose and why.)

6.2 The unknown distribution and the hypothesis

Story / analogy: the exam syllabus you never get to see

Nature has a giant, secret "question bank" $P(X, Y)$ that says which (patient, outcome) pairs are likely. Our 200 records are 200 questions drawn from that bank. Tomorrow a new patient walks in, which is a fresh question from the same bank. We never see the bank itself, only the questions we happened to draw. Learning means preparing for *unseen* questions using only the ones we have practised.

Formally, we assume each $(\mathbf{x}_i, y_i)$ is drawn from an **unknown** distribution $P(X, Y)$. Our goal is a function (also called a model, machine, or **hypothesis**) h such that, for a new pair $(\mathbf{x}, y) \sim P$,

$$\underbrace{h(\mathbf{x}) = y \ \text{ with high probability}}_{\text{classification}} \qquad \text{or} \qquad \underbrace{h(\mathbf{x}) \approx y \ \text{ in some optimal sense}}_{\text{regression}}.$$

The hypothesis class $\mathcal{H}$. We don't search over *all* possible functions. We first choose a family of candidates, the **hypothesis class** $\mathcal{H}$, for example all straight lines, all neural networks of a given shape, or all decision trees. Then we choose the best member $h \in \mathcal{H}$. The slides answer "How?" in two steps: (1) define $\mathcal{H}$; (2) evaluate each candidate and pick the best one. Step (2) needs a way to *score* candidates, and that score is the loss function.

Story / analogy: choosing a suit

$\mathcal{H}$ is the rack of suits in the shop (the *styles* you are willing to consider). Training is trying them on and choosing the one that fits best. If the rack only has children's sizes (a class that is too simple), no amount of trying on will give a good fit.

7 Loss functions: putting a number on "wrong"

A loss function $\mathcal{L}$ measures how badly h predicts y from $\mathbf{x}$, averaged over the data. Every loss below divides by n, so it is a *loss per example*. That makes a dataset of 200 comparable with a dataset of 2 million.

name	formula	penalty for one error $e = h(\mathbf{x}_i) - y_i$
0/1 loss	$\mathcal{L}_{0/1}(h) = \frac{1}{n}\sum_{i=1}^{n}\left(1 - \delta_{h(\mathbf{x}_i)-y_i}\right)$	0 if exactly right, 1 otherwise
squared loss (MSE)	$\mathcal{L}_{\text{sq}}(h) = \frac{1}{n}\sum_{i=1}^{n}\left(h(\mathbf{x}_i) - y_i\right)^2$	e^2 (grows quadratically)
root MSE (RMSE)	$\mathcal{L}_{\text{rms}}(h) = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(h(\mathbf{x}_i) - y_i\right)^2}$	$\sqrt{\text{MSE}}$, in the units of y
absolute loss (MAE)	$\mathcal{L}_{\text{abs}}(h) = \frac{1}{n}\sum_{i=1}^{n}\left\lvert h(\mathbf{x}_i) - y_i\right\rvert$	$\lvert e\rvert$ (grows linearly)

Here $\delta_k = 1$ if $k = 0$ and $\delta_k = 0$ otherwise, so $1 - \delta_{h(\mathbf{x}_i)-y_i}$ is simply "1 if we made a mistake". The 0/1 loss therefore *counts mistakes*, divided by n. All four losses are non-negative, and 0 means perfect on D.

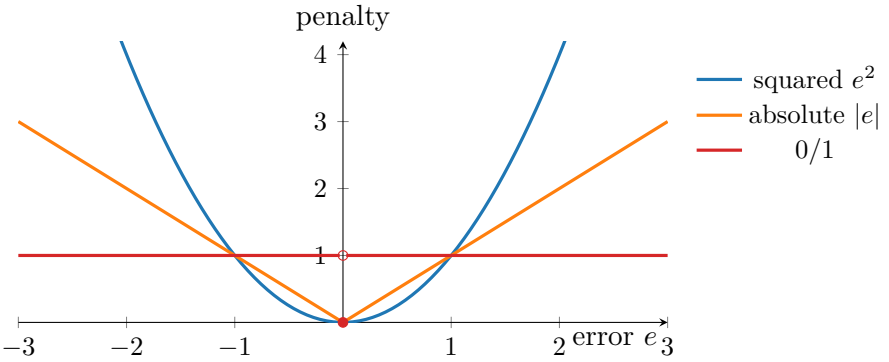


Figure 5: How each loss punishes a single error. Squared loss barely cares about small errors but explodes for big ones. Absolute loss is even-handed. 0/1 loss is flat almost everywhere, with a jump at $e = 0$.

Reading the pictures.

- **0/1 loss** is flat almost everywhere with a single jump, so it is *non-continuous and non-differentiable*.

  Story / analogy: the blindfolded hiker

  Training usually means walking downhill on the loss surface by feeling its slope. On a 0/1 landscape the ground is perfectly flat everywhere except at sudden cliffs. A blindfolded hiker feels no slope and has no idea which way to step. That is why 0/1 loss is great for *reporting* performance but rarely used for *training*.

- **Squared loss** is smooth (easy to optimise) but punishes big errors *quadratically*: a single error of 10 costs as much as a hundred errors of 1.
- **Absolute loss** grows linearly, so a few wild points (noise, outliers) can't dominate it. That is why the slide calls it "suited for noisy data".
- **RMSE** undoes the squaring, so it is measured in the *same units* as y. An RMSE of 3 on oxygen saturation means "typically off by about 3 percentage points". MSE would report 9 "squared percentage points", which nobody can interpret.

Mini example. Four patients' oxygen saturation (%), and one model's predictions:

patient	1	2	3	4
true y_i	97	94	99	90
predicted $h(\mathbf{x}_i)$	96	95	99	84
error e_i	-1	$+1$	0	-6

$$\text{MSE} = \tfrac{1+1+0+36}{4} = 9.5$$
$$\text{RMSE} = \sqrt{9.5} \approx 3.08$$
$$\text{MAE} = \tfrac{1+1+0+6}{4} = 2$$

Patient 4 alone contributes $36/38 \approx 95\%$ of the squared loss but only $6/8 = 75\%$ of the absolute loss.

Brainteaser 8 solution →

Suppose your hypothesis class is the laziest one possible: *constant* predictors $h(\mathbf{x}) = c$. The training labels are $\{1, 2, 3, 10\}$. (a) Which c minimises the squared loss? (b) Which c minimises the absolute loss? (c) Now change the 10 to 1000. How do your answers move, and what does

that say about "absolute loss is suited for noisy data"?

8 Learning, overfitting and the train/test split

8.1 Learning is optimisation

Once $\mathcal{H}$ and $\mathcal{L}$ are fixed, learning is an *optimisation problem*:

$$h^* = \arg\min_{h \in \mathcal{H}} \mathcal{L}(h).$$

(The slides write min, but what we really want is the *minimiser*, the machine itself, and that is what arg min denotes.) This finds an h with low loss *on our data*. But the data were drawn from an *unknown* P, so the real question is the one in red on the slides:

*How can we make sure h gives low loss on inputs that are **not** in D?*

8.2 The memoriser: the first glimpse of overfitting

Here is a model with *zero* training loss:

$$h(\mathbf{x}) = \begin{cases} y_i, & \text{if } \mathbf{x} = \mathbf{x}_i \text{ for some } (\mathbf{x}_i, y_i) \in D, \\ 0, & \text{otherwise.} \end{cases}$$

Story / analogy: the student who memorised the answer key

A student memorises last year's past paper word for word. On last year's questions they score 100%. The real exam has new questions, and on those they write "0" every time. The memoriser is perfect on D and useless on everything else. Doing brilliantly on the data you trained on while failing on new data is called **overfitting**.

8.3 The fix: hold data back

The answer is to *hide some data from yourself*. Split D into

- D_{TR}, the **training set** (typically 70, 80 or 90%), and
- D_{TE}, the **test set** (the remaining 30, 20 or 10%).

Learn on the training set only,

$$h^* = \arg\min_{h \in \mathcal{H}} \frac{1}{|D_{\text{TR}}|} \sum_{(\mathbf{x},y) \in D_{\text{TR}}} \mathcal{L}(\mathbf{x}, y \,|\, h),$$

and then report the **test error**

$$\epsilon_{\text{TE}} = \frac{1}{|D_{\text{TE}}|} \sum_{(\mathbf{x},y) \in D_{\text{TE}}} \mathcal{L}(\mathbf{x}, y \,|\, h^*)$$

as an honest estimate of performance on new data.

How to split. The split has to capture the variation in the distribution. Both halves should look like fresh draws from P.

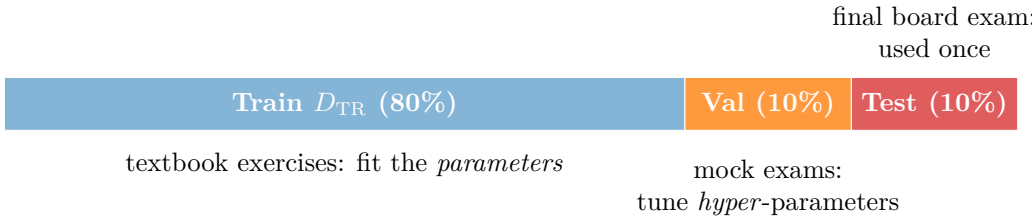
- Ordinary data: shuffle, then split at random (**i.i.d. sampling**).
- **Time series: split by time.** Train on the past and test on the future. A weather model that trains on next week's data and then "predicts" next week is cheating.

Common trap: the test set is a sealed envelope

You may use the test set **once**, after all decisions about the model are final. If you look at test error, tweak the model, and look again, the test set has quietly become part of training. Your reported number is then optimistic, in the same way that a student who has seen the final exam paper "scores well" in it.

8.4 Validation: the mock exam

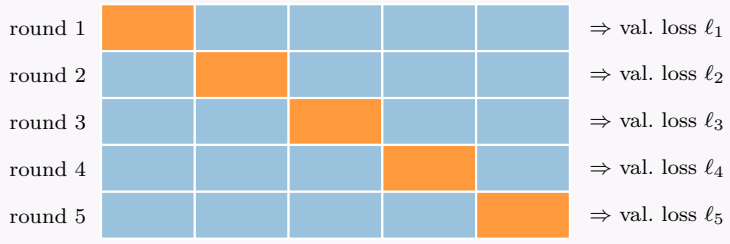
But we *do* need to make decisions. Which polynomial degree? Which k in k-NN? The answer is a third set:



- **Training set:** used for learning, i.e. fitting the *parameters* of the hypothesis (the polynomial coefficients, the weights of a network).
- **Validation set:** used to tune the **hyper-parameters**, the choices you make *before* training (polynomial degree, number of hidden units, k). If validation loss is high, change the model and try again. You may do this as often as you like.
- **Test set:** used *only* to assess the final, fully specified model.

Beyond the slides: k-fold cross-validation

With little data, a single 10% validation slice is noisy. **K-fold cross-validation** rotates the role of validation set through the training data and averages the K validation losses:



$$\text{CV loss} = \frac{1}{5}\sum_{j=1}^{5} \ell_j$$

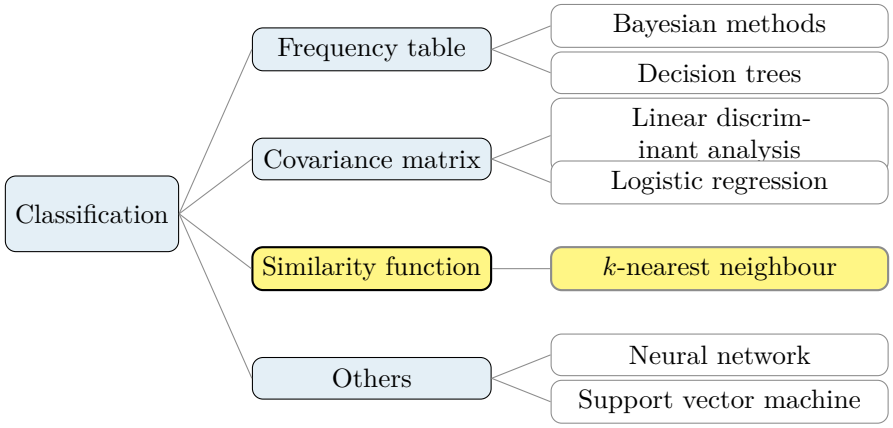
Every example is used for validation exactly once. The test set stays sealed throughout.

Brainteaser 9 solution →

Which set (train / validation / test) is being used in each situation, and is the use legitimate? (a) Choosing between $k = 3$ and $k = 7$. (b) Computing the neural-network weights by gradient descent. (c) Reporting "our model is 91% accurate" in a paper. (d) Noticing that test accuracy is low, adding features, and re-testing on the same test set.

9 A map of classifiers

Predicting a categorical output is called **classification**. The slides group the classic methods by the tool each one uses:



We now study the highlighted branch.

10 k-Nearest Neighbours: "tell me who your friends are"

10.1 The idea

Story / analogy: the new family on the street

A new family moves into your neighbourhood and you want to guess which cricket team they support. You know nothing about them, but you know that the three families living closest to them all support Lahore Qalandars. A good guess: Qalandars. That is the whole of k-NN: **similar inputs have similar labels**, so label a new point the way its closest known neighbours are labelled.

The algorithm, in three lines:

1. Given a new (test) point $\mathbf{x}$ whose class y we need,
2. find the **most similar** points in the training data,
3. classify $\mathbf{x}$ "like" those points (by majority vote).

The simplest case, **1-NN**, just copies the label of the single closest point. This makes it the "smart memoriser": on a training point it returns that point's label, and on a new point, instead of saying "0", it borrows the answer of the most similar training example.

This raises three questions, which the slides pose and which we answer in turn. *How do we measure similarity?* (distance metrics, §10.4); *how many neighbours?* (choice of k, §10.5); *how do we break disagreements?* (majority vote and ties, §10.6).

10.2 Formal definition

Let $D = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$ with $y_i \in \{1, \ldots, M\}$. For a query point $\mathbf{x}$:

- the **neighbour set** $S_{\mathbf{x}} \subseteq D$ contains the k training points closest to $\mathbf{x}$;
- the classifier outputs the most common label in it:

$$h(\mathbf{x}) = \text{mode}\{\, y_i : (\mathbf{x}_i, y_i) \in S_{\mathbf{x}} \,\} \qquad \text{(majority vote)}.$$

Regression version: average the neighbours' labels instead of voting,

$$h(\mathbf{x}) = \frac{1}{k} \sum_{(\mathbf{x}_i, y_i) \in S_{\mathbf{x}}} y_i, \qquad \text{or the inverse-distance weighted average} \qquad h(\mathbf{x}) = \frac{\sum_{S_{\mathbf{x}}} y_i / \operatorname{dist}(\mathbf{x}, \mathbf{x}_i)}{\sum_{S_{\mathbf{x}}} 1 / \operatorname{dist}(\mathbf{x}, \mathbf{x}_i)},$$

where the weighted version lets closer neighbours count for more.

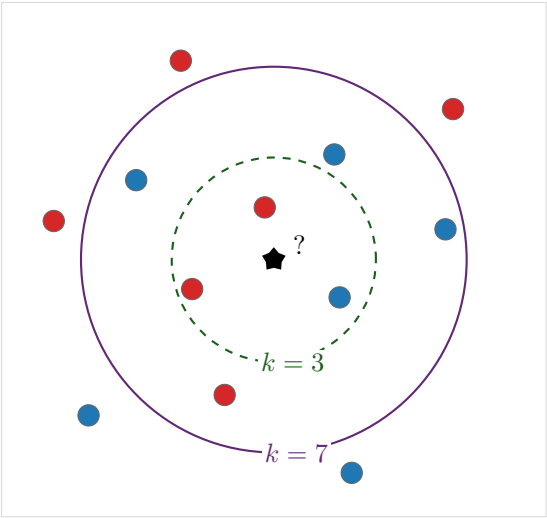


Figure 6: The same query point $\star$ gets different labels as k grows. The circles grow just until they contain k training points. (These are real distances: the seven nearest points are at $0.70, 1.00, 1.15, 1.60, 1.90, 2.10, 2.30$.)

10.3 What kind of learner is k-NN?

- **Instance-based ("lazy") learning.** There is no training phase that builds a model. k-NN just *stores* the training data, and all the work happens when a prediction is requested.

  Story / analogy: the open-book student

  Most models are like students who study all semester and walk into the exam with a compact summary in their heads (the parameters). k-NN is the student who does no studying at all and carries the entire library into the exam hall, looking up the most similar solved problem for every question. Preparation is instant, but the exam is slow and the bag is heavy.

- **Flexible.** It adapts to any new point by computing distances when needed, and it can draw very irregular class boundaries.
- **No distributional assumptions.** It never assumes the data are Gaussian, linear, or anything else.
- **Non-parametric.** There are no learned parameters (no weights to fit). The "model" *is* the dataset.
- **Hyper-parameters.** There are two: k (the number of neighbours) and the distance metric (the definition of "similar").
- **Cost grows with the data.** Both prediction time and storage grow with n (§10.7).

10.4 Measuring similarity: distance metrics

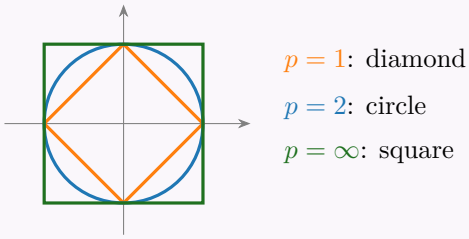
For numeric features, the natural choice is the Euclidean distance from Notes 01. The slides also give the choice for categorical features.

Beyond the slides: the Minkowski family

$$\text{dist}_p(\mathbf{x}, \mathbf{x}') = \Big(\sum_{j=1}^{d} |x_j - x'_j|^p \Big)^{1/p}$$

- $p = 2$: **Euclidean** distance (as the crow flies), the default;

- $p = 1$: **Manhattan** distance (walking along city blocks), $\sum_j |x_j - x'_j|$;
- $p \to \infty$: **Chebyshev** distance, $\max_j |x_j - x'_j|$ (only the worst coordinate counts).



These are the points at distance exactly 1 from the origin under each metric. A different metric gives a different shape of "neighbourhood", and so possibly different neighbours.

Categorical features: Hamming distance. For features such as blood group or smoker/non-smoker, differences like "O^+ minus A^+" mean nothing, so we count *mismatches* instead:

$$\text{dist}(\mathbf{x}, \mathbf{x}') = \sum_{j=1}^{d} (1 - \delta_{x_j - x'_j}) = \#\{\, j : x_j \neq x'_j \,\}.$$

Example: $\mathbf{x}$ = (A^+, smoker, male, urban) and $\mathbf{x}'$ = (O^+, smoker, female, urban) differ in 2 positions, so dist = 2. (It is the 0/1 loss again, applied feature by feature!)

Practical issue: features on different scales. A distance squashes a d-dimensional difference into one number, so every feature should be on a comparable scale. Otherwise the feature with the biggest numbers wins by default.

	age (years)	monthly income (PKR)	raw Euclidean distance to Q
query Q	25	80,000	–
person A	60	80,500	$\sqrt{35^2 + 500^2} \approx 501$
person B	26	82,000	$\sqrt{1^2 + 2000^2} \approx 2000$

On raw numbers, the 60-year-old is "closer" to the 25-year-old than the 26-year-old is. Income, measured in thousands, drowns out age. **Solution: normalise the data.** Two common ways:

$$\text{min–max: } x_j \mapsto \frac{x_j - \min_j}{\max_j - \min_j} \in [0, 1], \qquad \text{z-score: } x_j \mapsto \frac{x_j - \mu_j}{\sigma_j}.$$

If age has $\sigma = 12$ years and income has $\sigma = 20{,}000$ PKR, the z-scored distances become A : $\sqrt{2.92^2 + 0.025^2} \approx 2.92$ and B : $\sqrt{0.083^2 + 0.1^2} \approx 0.13$. Now B is the nearest neighbour, as common sense says.

Common trap: normalisation leaks

Compute μ_j, σ_j (or $\min_j, \max_j$) on the **training set only**, then apply the same numbers to validation and test points. Computing them on all the data lets information from the sealed envelope leak into training.

Brainteaser 10 solution →

Two patients are described by (blood group, smoker?, diabetic?, city): $\mathbf{x}$ = (B^+, yes, no, Lahore) and $\mathbf{x}'$ = (B^+, no, no, Karachi). (a) What is their Hamming dis-

tance? (b) What is the largest possible Hamming distance between two such patients? (c) Why would Euclidean distance on the codes Lahore = 1, Karachi = 2, Islamabad = 3 be a bad idea?

10.5 Choosing k: between paranoia and blindness

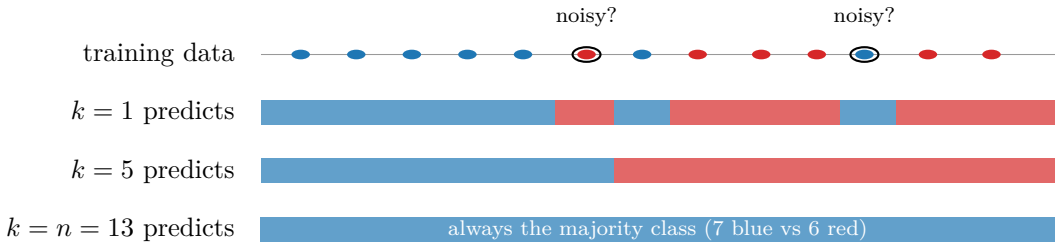


Figure 7: A one-dimensional feature with two suspicious labels (circled). With $k = 1$ every noisy point carves out its own little island, which gives a jumpy, high-variance classifier. With $k = 5$ the islands are outvoted and one clean boundary remains. With $k = n$ the classifier ignores $\mathbf{x}$ completely.

- $k = 1$**:** sensitive to noise, **high variance**. A single mislabelled point flips predictions in its neighbourhood. Increasing k makes the algorithm *less sensitive to noise*.
- $k = n$**:** every query has the same neighbours (everyone), so the prediction is always the overall majority class. This is maximally stable but blind to the input. *Decreasing* k lets the model capture the *finer structure* of the space.

Story / analogy: asking for directions

Lost in a new city, you ask one passer-by ($k = 1$). If they happen to be wrong, you are wrong. You ask the entire city and take the majority answer ($k = n$). You get the same generic reply ("go to the Mall Road!") no matter where you are standing. The sweet spot is asking a handful of people standing *near you*.

How do we choose k? Learn it from data.

1. Split the data into training and validation (or, better, use 5- or 10-fold cross-validation).
2. For $k = 1, 2, 3, \ldots$ compute the validation loss, using only the training part to find neighbours.
3. Choose the k with the smallest validation loss.

Choosing k is *the only learning k-NN does*.

10.6 Practical issues

- **Use odd k for binary classification.** With two classes and odd k, a vote can never split evenly.
- **Ties** (possible with even k or with more than two classes): use prior information (e.g. pick the more common class), or fall back to the 1-NN or $(k-1)$-NN classifier to decide.
- **Missing values:** fill a missing feature with that feature's average value (computed on the training set) before computing distances.

Brainteaser 11 solution →

(a) With $M = 3$ classes, does an odd k guarantee no ties? (b) In Figure 6, what does $k = 2$ predict, and how would the "fall back to $k - 1$" rule resolve it? (c) What does $k = 12$ (all the

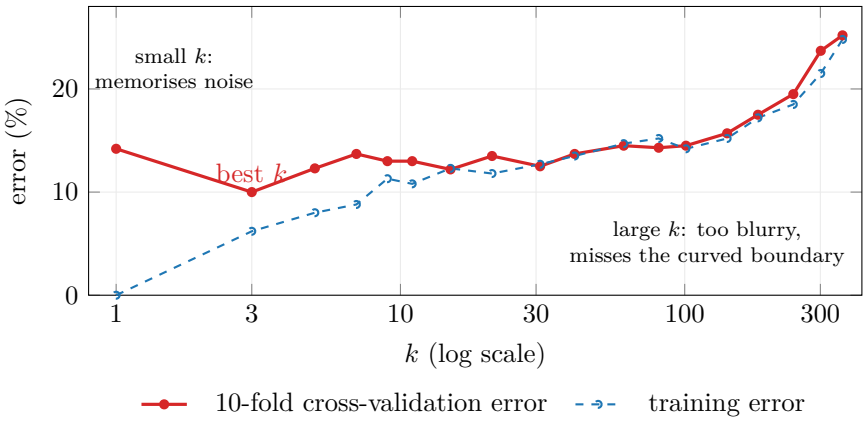


Figure 8: A real run of k-NN on 400 points of a noisy two-moons dataset. Training error is 0 at $k = 1$ (each point is its own nearest neighbour, so this is the memoriser again), but that tells you nothing. Validation error is what counts. It is lowest for a small, but not the smallest, k and rises steadily as k grows.

points) predict?

10.7 Storage and time complexity

To classify *one* test point with n training points in d dimensions:

step	how	cost
1. distance to every training point	n distances, each touching d numbers	$\mathcal{O}(dn)$
2. find the k closest	find the k-th smallest distance (median of medians)	$\mathcal{O}(n)$
	collect the points at or below that distance	$\mathcal{O}(n)$
3. majority vote	count labels among k points	$\mathcal{O}(k)$
total per query	dominated by step 1	$\boldsymbol{\mathcal{O}(dn)}$
storage	must keep the whole training set	$\boldsymbol{\mathcal{O}(dn)}$

Step 2 does *not* sort. Sorting all n distances would cost $\mathcal{O}(n \log n)$. We only need the k smallest, and a selection algorithm finds the k-th smallest in linear time.

Key idea

Most models: slow training, fast prediction. k-NN: *free* training, *expensive* prediction. That trade-off is the price of being "lazy".

Brainteaser 12 solution →

A face-recognition kNN stores $n = 10^6$ faces, each a $d = 128$-dimensional embedding. (a) Roughly how many arithmetic operations does one query cost? (b) How much memory is needed at 4 bytes per number? (c) If the dataset doubles, what happens to the query time, and what happens to the training time?

Watch next: StatQuest, *K-nearest neighbors, clearly explained*.

11 Grading a classifier honestly

11.1 Accuracy and its trap

The 0/1 loss averaged over the test set is the **misclassification rate**. It estimates the probability that a random point is classified wrongly. Its complement is **accuracy**:

$$\text{Accuracy} = 1 - \mathcal{L}_{0/1}.$$

Story / analogy: the lazy umpire

In one innings there are 315 deliveries and 15 of them are no-balls. A "model" answers the question "will this ball be legal?" with **Yes** every single time, without ever looking at the bowler. Its accuracy is $300/315 \approx \mathbf{95.2}\%$. It sounds excellent, yet it has never caught a single no-ball, which was the whole point of the model.

Common trap: imbalanced classes

When one class dominates (95.24% legal balls vs 4.76% no-balls), a model can score high accuracy just by always predicting the majority. Accuracy is *not meaningful* for imbalanced or skewed classes. We need to count *which kinds* of mistakes are made.

11.2 TP, TN, FP, FN

Take a binary problem with $\mathcal{Y} = \{0, 1\}$, where 1 is called *positive* and 0 *negative*. The true labels y are the *ground truth* (also called gold labels or gold standard), and the classifier outputs $\hat{y} = h(\mathbf{x})$.

name	truth y	prediction $\hat{y}$
TP true positive	1	1
TN true negative	0	0
FP false positive (false alarm)	0	1
FN false negative (a miss)	1	0

Key idea: a decoding trick

Read the name backwards. The *second* word (Positive/Negative) is *what the model said*. The *first* word (True/False) is *whether it was right*. So "false negative" means "the model said negative, and that was wrong": a sick patient sent home.

11.3 The confusion matrix

Arrange the four counts in a 2×2 table. As on the slides, **rows are predictions** $\hat{y}$ and **columns are the truth** y.

		actual	
		1 (pos)	0 (neg)
predicted	1 (pos)	**TP**	**FP**
	0 (neg)	**FN**	**TN**

TP + FP = predicted positives

FN + TN = predicted negatives

$P = \text{TP} + \text{FN}$ actual positives

$N = \text{FP} + \text{TN}$ actual negatives

The diagonal (green) holds the correct predictions and the off-diagonal (red) holds the mistakes.

Reading the slides carefully

On the confusion-matrix slide the total of the negative column is printed as "N = P + TN". It should read $N = \text{FP} + \text{TN}$: all the actually-negative cases, whether the model got them right or wrong. Also be aware that many textbooks and `scikit-learn` put the *truth on the rows* and the predictions on the columns, which transposes the table. Always check the axis labels.

Two running examples (from the slides).

Heart disease (1 = disease)

	$y = 1$	$y = 0$	
$\hat{y} = 1$	100	10	110
$\hat{y} = 0$	5	50	55
	$P = 105$	$N = 60$	165

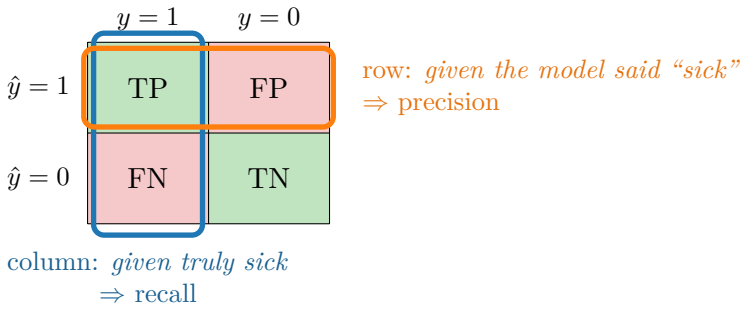
Cricket (1 = legal ball, 0 = no-ball)

	$y = 1$	$y = 0$	
$\hat{y} = 1$	298	7	305
$\hat{y} = 0$	2	8	10
	$P = 300$	$N = 15$	315

Read the cricket table in words. The model flagged 10 balls as no-balls, and 8 of those really were (TN = 8), while 2 were legal balls wrongly flagged (FN = 2). It missed 7 real no-balls, calling them legal (FP = 7). Notice that here "positive" means *legal ball*, because that is class 1, so catching a no-ball counts as a *true negative*. Which class you call "positive" is a choice, and it matters for some metrics (Brainteaser 16).

11.4 The metrics: rows versus columns

Every metric is a fraction of one row or one column of the confusion matrix. The **denominator tells you which world you are standing in**.



metric	formula	question it answers
Accuracy	$\frac{\mathrm{TP}+\mathrm{TN}}{P+N}$	Overall, how often is it right?
Error rate	$\frac{\mathrm{FP}+\mathrm{FN}}{P+N}$	Overall, how often is it wrong?
column metrics (condition on the truth)		
Sensitivity / Recall / TPR S_e	$\frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}} = \frac{\mathrm{TP}}{P}$	Of the truly positive, how many did we catch?
Specificity / TNR S_p	$\frac{\mathrm{TN}}{\mathrm{TN}+\mathrm{FP}} = \frac{\mathrm{TN}}{N}$	Of the truly negative, how many did we clear?
False positive rate FPR	$\frac{\mathrm{FP}}{\mathrm{TN}+\mathrm{FP}} = 1 - S_p$	Of the truly negative, how many false alarms?
row metrics (condition on the prediction)		
Precision	$\frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}}$	When it says positive, how often is it right?
Negative predictive value	$\frac{\mathrm{TN}}{\mathrm{TN}+\mathrm{FN}}$	When it says negative, how often is it right?

Story / analogy: fishing with a net

You cast a net into a lake to catch fish (the positives). The lake also has old boots in it (the negatives).

- **Recall:** what fraction of *all the fish in the lake* ended up in your net?
- **Precision:** what fraction of *what is in your net* is actually fish?

A giant net that scoops up the whole lake has perfect recall and terrible precision. A tiny net that catches exactly one fish has perfect precision and terrible recall.

Heart-disease example, all the numbers.

Accuracy	$(100+50)/165$	$= 0.91$	
Error rate	$(10+5)/165$	$= 0.09$	
Sensitivity (recall)	$100/105$	$= 0.95$	(95% of sick patients are detected)
FPR	$10/60$	$= 0.17$	(17% of healthy patients get a false alarm)
Specificity	$50/60$	$= 0.83$	
Precision	$100/110$	$= 0.91$	(91% of "disease" verdicts are correct)
NPV	$50/55$	$= 0.91$	

Brainteaser 13 solution →

Compute accuracy, sensitivity, specificity and precision for the cricket table. Which single number exposes how poor this model is at its real job of catching no-balls?

11.5 When to use which?

Decide which *mistake is more expensive*, then watch the metric that punishes that mistake.

- **Disease detection: we do not want FN.** Sending a sick patient home can be fatal, while a false alarm only costs a follow-up test. Watch **sensitivity** $\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$.
- **Fraud detection (as on the slide): we do not want FP.** If "positive" means "flag as fraud", every false positive blocks an honest customer's card, and enough of those drive customers away. Watch **specificity** $\mathrm{TN}/(\mathrm{TN}+\mathrm{FP})$ and **precision** $\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$.

(Spam filtering has the same flavour as fraud: a real e-mail sent to spam, an FP, is worse than one spam e-mail reaching the inbox.)

11.6 Trade-offs: the threshold slider

Most classifiers output a *score* ("how sick does this patient look?") and then compare it with a **threshold**. Moving that one slider trades one kind of error for the other.

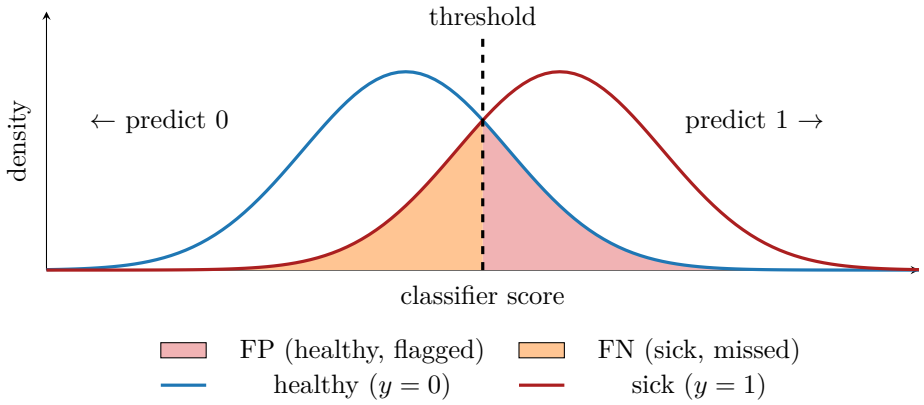


Figure 9: Scores of healthy and sick patients overlap, so no threshold is perfect. Slide the threshold **left**: fewer misses (sensitivity ↑) but more false alarms (specificity ↓). Slide it **right**: the reverse.

The two extreme doctors (from the slides).

- **The alarmist** declares *everyone* sick (threshold far left). Then $S_e = 1$, since no sick patient is missed, but $\mathrm{TN} = 0$, all the healthy patients become FPs, $S_p = 0$, and precision drops to $P/(P+N)$. For the heart data that is $105/165 = 0.64$.
- **The optimist** declares *everyone* healthy (threshold far right). Then $S_p = 1$ but $S_e = 0$: every sick patient is sent home.

Ideally $S_e = S_p = 1$, but with overlapping classes that is unrealistic. Both precision vs recall and sensitivity vs specificity are *trade-offs*, so we need tools that look at the whole slider at once.

11.7 The ROC curve and AUC

Slide the threshold from far right to far left. At each position, plot one point: $(\mathrm{FPR},\ \mathrm{TPR}) = (1 - S_p,\ S_e)$. The path traced out is the **ROC curve** (receiver operating characteristic).

- **Top-left corner** $(0, 1)$: the best possible classifier, with $S_e = S_p = 1$.
- **The diagonal** is the "line of no discrimination". A classifier that flips a biased coin lands on it, and it has no power.

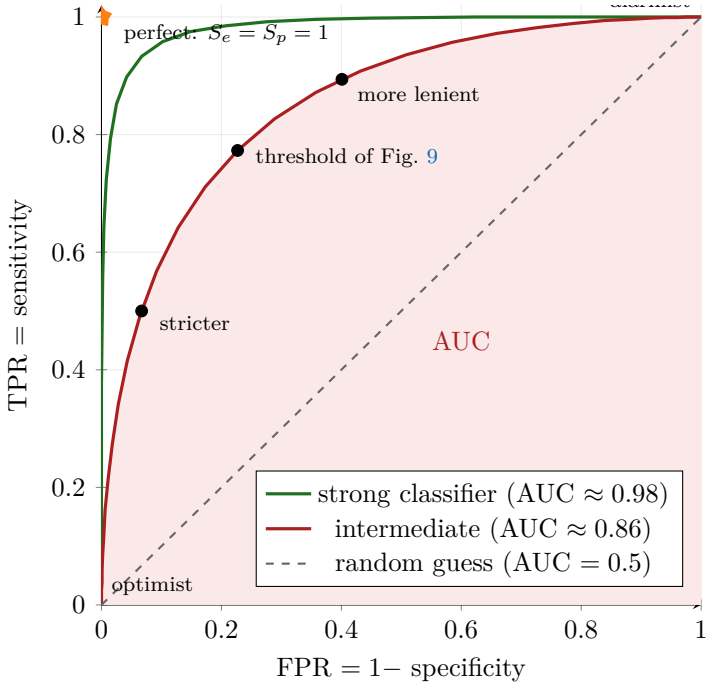


Figure 10: ROC curves. Every point on a curve is one threshold. The alarmist sits at $(1, 1)$ and the optimist at $(0, 0)$, so *every* classifier's curve joins those two corners. The question is how close it bends to the top-left star. The red-shaded region is the AUC of the intermediate classifier.

- **AUC** (Area Under the ROC Curve) compresses the whole curve into one number measuring the power of the classifier: 1 is perfect and 0.5 is random guessing.

Beyond the slides: what AUC really means

AUC has a lovely probabilistic reading: *pick one random sick patient and one random healthy patient. AUC is the probability that the classifier gives the sick one the higher score.* For the intermediate curve above, that probability is 0.86. This is why AUC doesn't depend on any single threshold. It measures how well the classifier *ranks* positives above negatives.

Brainteaser 14 solution →

A classifier's ROC curve lies entirely *below* the diagonal, with AUC $= 0.2$. Is it useless? What simple change would turn it into a good classifier?

11.8 F1-score: one number for precision and recall

Higher recall can usually be bought with lower precision, as the alarmist shows. We want one number that is high only when *both* are high. The slides list several combined measures: the F1 score, Matthews' correlation coefficient, 11-point average precision and the breakeven point. We study the first two.

Definition: F-scores

The **weighted harmonic mean** of precision and recall:

$$F_\beta = \frac{1+\beta^2}{\dfrac{1}{\text{Precision}} + \dfrac{\beta^2}{\text{Recall}}}, \qquad \beta \geq 0.$$

For $\beta = 1$ this is the plain harmonic mean, the **F1-score**:

$$F_1 = \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2\,\text{TP}}{2\,\text{TP} + \text{FP} + \text{FN}}.$$

$\beta > 1$ weights recall more (e.g. F_2 for disease screening), and $\beta < 1$ weights precision more (e.g. $F_{0.5}$ for fraud flags). For the heart data: $F_1 = 200/215 = 0.93$, $F_2 = 0.94$, $F_{0.5} = 0.92$.

Why the harmonic mean and not the ordinary average?

Story / analogy: the round trip

You drive 60 km to Islamabad at 30 km/h and drive back at 60 km/h. Your average speed is *not* 45 km/h. The trip took $2 + 1 = 3$ hours for 120 km, so it is 40 km/h, which is the harmonic mean of 30 and 60. The slow leg eats up more of the time, so it dominates the result. In the same way, F1 is dragged down by whichever of precision or recall is *worse*. You cannot hide a terrible recall behind a great precision.

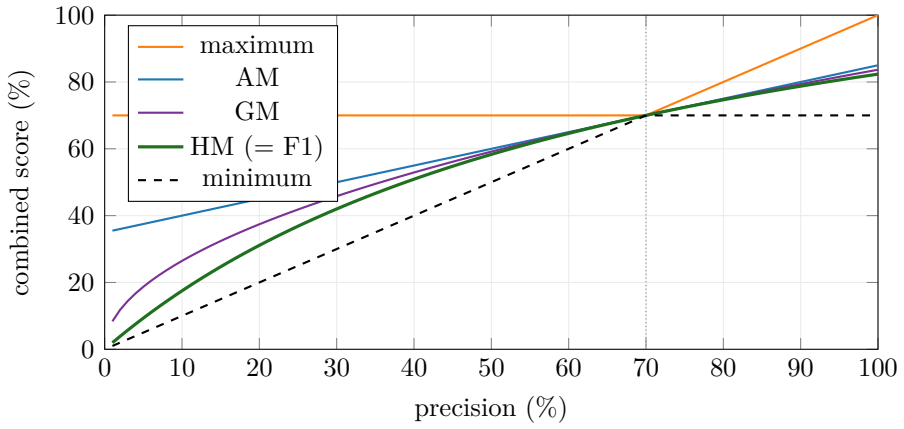


Figure 11: Recall is fixed at 70% and precision varies. At every point, min ≤ HM ≤ GM ≤ AM ≤ max. The harmonic mean hugs the *minimum*. At 5% precision the arithmetic mean still reports a respectable 37.5%, while F1 honestly reports 9.3%.

For two positive numbers, HM ≤ GM ≤ AM. The HM is the *conservative* average: it penalises an imbalanced model the most, and an improvement in HM implies an improvement in AM and GM.

Brainteaser 15 solution →

A fraud model flags just one transaction, and it really is fraud. There were 50 frauds in total. Compute precision, recall, their arithmetic mean, and F_1. Which of the two averages tells the truth about this model?

11.9 Matthews correlation coefficient (MCC)

Beyond the slides: MCC: the metric that uses all four cells

F1 never looks at TN at all. Look at its formula. MCC uses the whole confusion matrix:

$$\text{MCC} = \frac{\text{TP} \cdot \text{TN} - \text{FP} \cdot \text{FN}}{\sqrt{(\text{TP} + \text{FP})(\text{TP} + \text{FN})(\text{TN} + \text{FP})(\text{TN} + \text{FN})}} \in [-1, 1].$$

It is exactly the *correlation coefficient* (Notes 01) between the vector of true labels and the vector of predicted labels. +1 means perfect prediction, 0 means no better than chance, and

-1 means perfectly wrong. (If a factor in the denominator is 0, as for a model that always predicts one class, MCC is defined as 0.)

	accuracy	F1 (class 1 positive)	MCC
heart disease	0.91	0.93	0.80
cricket model	0.97	0.99	0.64
cricket "always Yes"	0.95	0.98	0

Accuracy and F1 both rate the useless "always Yes" model above 0.95, because the positive class is the *majority* class. MCC scores it 0, which is correct.

Brainteaser 16 solution →

In the cricket example, relabel so that *no-ball* is the positive class (swap the roles of 0 and 1). Recompute F1. Does MCC change? What does this teach you about reporting F1 on imbalanced data?

Watch next: StatQuest, *The confusion matrix* and *ROC and AUC, clearly explained*.

12 Summary: the big picture

Ten sentences to remember.

1. A matrix mixes its columns: $A\mathbf{x} = x_1\mathbf{a}_1 + \cdots + x_n\mathbf{a}_n$.
2. Rank = number of genuinely independent columns = number of independent rows = number of pivots, and it is at most $\min(m, n)$.
3. The column space is the menu of reachable outputs, and $A\mathbf{x} = \mathbf{b}$ is solvable exactly when $\mathbf{b}$ is on the menu.
4. The null space is the set of ghost recipes that the machine maps to zero. Each one records a redundancy among the columns.
5. Rank + nullity = number of columns: every input dimension either survives or is crushed.
6. Supervised learning picks $h \in \mathcal{H}$ to minimise a loss, hoping it generalises to the unseen distribution P.
7. Low training loss is not enough (remember the memoriser). Judge models on held-out data, tune on validation, and open the test set once.
8. k-NN votes among the k closest training points. Small k is noisy, large k is blurry, so pick k by validation and normalise the features first.
9. Accuracy lies on imbalanced data. Decide which mistake is costly and read the matching row or column of the confusion matrix.
10. ROC/AUC summarises every threshold at once. F1 (a harmonic mean) punishes imbalance between precision and recall. MCC uses all four cells.

Metric cheat-sheet (rows $= \hat{y}$, columns $= y$)		
Recall / Sensitivity / TPR	$\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$	"did we catch the positives?"
Specificity / TNR	$\mathrm{TN}/(\mathrm{TN}+\mathrm{FP})$	"did we clear the negatives?"
FPR	$\mathrm{FP}/(\mathrm{FP}+\mathrm{TN}) = 1 - S_p$	"false-alarm rate"
Precision	$\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$	"can we trust a positive verdict?"
NPV	$\mathrm{TN}/(\mathrm{TN}+\mathrm{FN})$	"can we trust a negative verdict?"
F1	$2\mathrm{TP}/(2\mathrm{TP}+\mathrm{FP}+\mathrm{FN})$	harmonic mean of prec. & recall
MCC	$\dfrac{\mathrm{TP}\cdot\mathrm{TN}-\mathrm{FP}\cdot\mathrm{FN}}{\sqrt{(\mathrm{TP}+\mathrm{FP})(\mathrm{TP}+\mathrm{FN})(\mathrm{TN}+\mathrm{FP})(\mathrm{TN}+\mathrm{FN})}}$	correlation of y and $\hat{y}$

Solutions to the brainteasers

Solution 1 (asked on p. 2). (a) No. Every column has green component 0, so every mixture $A\mathbf{x}$ has green component 0, while yellow has green = 1. Yellow is not in $\mathcal{C}(A)$. (b) $\mathbf{x} = (1, 1, -1)^\top$: red + blue − purple $= \mathbf{0}$. Any multiple $t(1, 1, -1)^\top$ also works, so there is a whole line of such recipes, and that line is $\mathcal{N}(A)$.

Solution 2 (asked on p. 5). B_1: every row is a multiple of $(2, 4, -6)$ (row 2 is $\frac{1}{2}$ of row 1 and row 3 is $-\frac{3}{2}$ of it), so the rank is 1, a *line*. $B_2 = I$: rank 3, the *volume*. B_3: column 3 = column 1 + column 2 and the first two are independent, so the rank is 2, a *plane* (the x_1x_2-plane). B_4: rank 0. It squishes everything to a single point, the origin, which is a fourth picture not drawn in the figure.

Solution 3 (asked on p. 5). $\mathbf{u}\mathbf{v}^\top$ is 5×4. Writing it as the product of a 5×1 matrix and a 1×4 matrix, (P2) gives rank $\leq \min(1, 1) = 1$. It is non-zero, so the rank is exactly 1: every column is a multiple of $\mathbf{u}$. It stores 20 numbers but needs only $5 + 4 = 9$. That saving is the seed of low-rank compression.

Solution 4 (asked on p. 6). (a) Yes: it *is* column 2 (recipe $(0, 1, 0)^\top$). (b) Yes: column 1 + column 3 $= (1, 2, 3, 4) + (2, 3, 4, 5) = (3, 5, 7, 9)$ ✓, so the recipe $(1, 0, 1)^\top$ works, and so does $(2, 1, 0)^\top$, since $2\mathbf{a}_1 + \mathbf{a}_2 = (3, 5, 7, 9)$. Infinitely many recipes exist because $\mathcal{N}(A) \neq \{\mathbf{0}\}$. (c) No. A mixture $x_1\mathbf{a}_1 + x_2\mathbf{a}_2$ (column 3 adds nothing new) has entries $x_1 + x_2$, $2x_1 + x_2$, $3x_1 + x_2$, $4x_1 + x_2$. The first two being 0 forces $x_1 = x_2 = 0$, but then the last entry is $0 \neq 1$. So $(0, 0, 0, 1)^\top$ is off the plane.

Solution 5 (asked on p. 9). (a) $\dim \mathcal{N}(W) = 7 - 3 = 4$. (b) Yes: $\mathcal{C}(W) \subseteq \mathbb{R}^3$ has dimension 3, so it is all of $\mathbb{R}^3$. Every $\mathbf{b}$ is reachable, in infinitely many ways. (c) A 7×3 matrix can have rank 3 and then nullity $3 - 3 = 0$, so yes. A 3×7 matrix has rank at most 3, so its nullity is at least $7 - 3 = 4 > 0$. A wide matrix *always* forgets something.

Solution 6 (asked on p. 9). (a) $(3, 6)^\top = 3 \cdot (1, 2)^\top$ lies on the blue line. One solution is $\mathbf{x}_p = (3, 0)^\top$, and all solutions are $(3, 0)^\top + t(2, -1)^\top$, $t \in \mathbb{R}$, a line parallel to $\mathcal{N}(M)$. (b) $(3, 5)^\top$ is not on the blue line $\mathcal{C}(M)$ (it would need $5 = 2 \cdot 3$), so there are *no* solutions.

Solution 7 (asked on p. 11). (a) Regression, since temperature is continuous. (b) Binary classification. (c) Multi-class classification with $M = 8$ blood groups. (d) Runs are integers $0, 1, \ldots, 36$. You *could* treat them as 37 classes, but the classes are ordered (5 runs is "closer" to 6 than to 20), so regression is usually the better choice, rounding if needed. Either choice is fine if you justify it.

Solution 8 (asked on p. 12). (a) Minimise $\sum(c - y_i)^2$ by setting the derivative $2\sum(c - y_i)$ to 0, which gives $c =$ the *mean* $= 4$. (b) $\sum |c - y_i|$ is minimised by the *median*. Here any $c \in [2, 3]$ works, since moving c right of 3 moves it away from three points and toward only one. (c) The mean jumps to 251.5, while the median stays in $[2, 3]$. A single wild label drags the squared-loss answer far away but barely moves the absolute-loss answer. That is exactly what "absolute loss is suited for noisy data" means.

Solution 9 (asked on p. 14). (a) Validation (or cross-validation): legitimate, since this is hyper-parameter tuning. (b) Training: legitimate. (c) Test: legitimate, *if* it was opened once, at the very end. (d) Test, and *not* legitimate. The test set has now been used for model selection, so the reported accuracy is optimistic. The added features should have been judged on the validation set.

Solution 10 (asked on p. 17). (a) They differ in "smoker" and "city", so the distance is 2. (b) 4, the number of features. (c) The codes invent a geometry that isn't there: Islamabad would be "twice as far" from Lahore as Karachi is, purely because of an arbitrary numbering. Categories are either equal or not, and Hamming distance respects that.

Solution 11 (asked on p. 18). (a) No. With $k = 3$ and one neighbour from each class, the vote is 1–1–1. Odd k only prevents ties when there are two classes. (b) The two nearest are red (0.70) and blue (1.00), a 1–1 tie. Falling back to $k - 1 = 1$ gives **red**. (c) There are 6 red and 6 blue points: another tie! This shows that $k = n$ just returns the overall class balance, and that even k invites ties.

Solution 12 (asked on p. 19). (a) About $d \cdot n = 128 \times 10^6 \approx 1.3 \times 10^8$ multiply-adds, plus $\mathcal{O}(n)$ for selection. (b) $128 \times 10^6 \times 4$ bytes ≈ 512 MB. (c) Query time doubles, since it is linear in n. Training time stays at essentially zero, since k-NN only stores the data. (In practice, approximate-nearest-neighbour indexes are used to speed up queries.)

Solution 13 (asked on p. 22). Accuracy $= (298+8)/315 = 0.971$. Sensitivity $= 298/300 = 0.993$. Specificity $= 8/15 = 0.533$. Precision $= 298/305 = 0.977$. The specificity exposes the problem: of the 15 real no-balls, the model catches only 8, about half. Everything else looks superb only because legal balls are so common.

Solution 14 (asked on p. 24). It is not useless. It is *reliably wrong*, which is information. Flip every prediction (or negate the score), and every point (FPR, TPR) becomes $(1-\text{FPR}, 1-\text{TPR})$. The curve is reflected through the centre, and the AUC becomes $1 - 0.2 = 0.8$. A truly useless classifier is one *on* the diagonal.

Solution 15 (asked on p. 25). Precision $= 1/1 = 1$ and recall $= 1/50 = 0.02$. AM $= 0.51$, which sounds like "half decent". $F_1 = \dfrac{2(1)(0.02)}{1.02} \approx 0.039$, which correctly reports that the model is nearly worthless. The harmonic mean tells the truth.

Solution 16 (asked on p. 26). With no-ball as positive, $\text{TP}' = 8$, $\text{FN}' = 7$ (no-balls called legal), $\text{FP}' = 2$ and $\text{TN}' = 298$. Then $F_1' = \dfrac{2 \cdot 8}{2 \cdot 8 + 2 + 7} = \dfrac{16}{25} = 0.64$, far below the 0.985 we had before. MCC is unchanged at 0.64: swapping the classes swaps TP with TN and FP with FN, and the formula is symmetric under that swap. Lesson: F1 depends on which class you call positive. On imbalanced data, make the *rare, important* class the positive one, or report MCC as well.